

Calculation & Simulation & Methods

The aim of this study is to analyze the performance of the parallel hybrid e-bike shown in Figure 1. The maximum engine power required by the vehicle and its regeneration potential will be calculated using basic vehicle dynamics models.

Driving cycle

This is a the raw data used to create driving cycle.To clean and process the chaotic, raw real-world data, we smoothed and downsampled it from 5 Hz to 1 Hz(1 data point per second), applied filters, and removed outliers .Our data includes detailed analyses of



Figure 1 – Driving Cycle

volunteer drivers recorded in Massachusetts, USA, including their vehicle, age, and average slope. The slopes are arranged into three main lines: -1, 0 and 1 and the true average slope becomes meaningful data.

Parameters of Vehicle

In this project, the parameters used are based on the Commuter S250 electric bicycle. We are designing it as a typical "Urban/Trekking E-Bike" that complies with European standards.

Parameter	Value	Unit	Rationale
Total Mass m	125	kg	Assumption: 100kg rider + 25kg e-bike.
Gravity Acceleration g	9.81	m/s ²	Standard physical constant.
Rolling Coefficient C_{rr}	0.008	-	Typical tire/road coefficient for asphalt.
Air Density ρ	1.225	kg/m ³	Standard air density at sea level, 20°C.
Frontal Area A_f	0.7	m ²	Assumption for Samim Unlusoy. $0.8 \times width(0.5m) \times height(1.75m)$
Drag Coefficient C_D	0.85	-	Aerodynamic shape factor for a semi-recumbent position.

Wheel Radius r_w	0.35	m	Represents a 28-inch (700c) wheel size (≈ 0.35 m radius).
Wind Speed v_{air}	0	m/s	No einf. $v_g + v_{air} = v_g$
Asistance cons. $K_{asistance}$	1.5	-	Based on tour mod
Rotary Mass Factor ϕ	1.05	-	Typically urban bike
Efficiency Constant $n_t n_m$	0.8 0.9	-	Direct gear(Samim Unlusoy's note)
Gradability θ	$\mp \%3$	%	Gradibilty Massachusetts ABD $\mp 1.72^\circ$

Table 1 - Parameters

Architecture and Controller Design & Mathematical Models

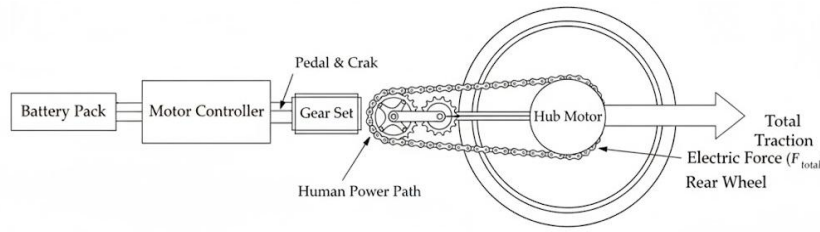


Figure 2 - Mechanism

According to regulations the assistance constant determines how much power our engine should produce. While $P_{motor}(1 + K_{asistance}) = P_{b-out}$ represents the power delivered to the wheel, the expression $\frac{T_m i_t}{K_{asistance}} + T_m = T_w$ represents the torque going to the wheel. Similarly, the power coming from regeneration can be calculated. Below are the forces that prevent us.

$$F_d = \frac{1}{2} C_d \rho_{air} A (v_g + v_{air})^2 \quad \& \quad F_{hill} = Mg \sin(\theta) \quad \& \quad F_{rolling} = Mg f_r$$

$$F \times v = P \quad \& \quad F \times r_w = T$$

Analysis of Motor Power: Losses (Traction) & Gains (Regeneration)

When vehicle dynamics are calculated, the calculations below are essentially performed, and the reason for the vehicle's acceleration or deceleration in the driving cycle clearly emerges. If the vehicle is decelerating, the reason for this is either that it is going uphill or it is performing regen. This means both of these formulas cannot take a positive value at the same time. The easiest way to apply these formulas is to look at the sign of the required Power.

$$P_{b-out} = \frac{V}{n_t n_m} \left[\overbrace{M \phi \frac{dV}{dt}}^{\text{Rotary mass}} + \overbrace{\frac{1}{2} C_d \rho_{air} A (v_g + v_{air})^2}^{\text{Air resistance}} + \overbrace{Mg \sin(\theta)}^{\text{Hill resistance}} + \overbrace{Mg \times (f_r)}^{\text{Rolling resistance}} \right]$$

$$P_{b-in} = \alpha V \times n_t n_m \left[-M \varphi \frac{dv}{dt} - \frac{1}{2} C_d \rho_{air} A (v_g + v_{air})^2 + M g \sin(\theta) - M g \times 0.01 \times \left(1 + \frac{v}{160} \right) \right]$$

To find the required max power, motor selection must be made by considering the $P_{motor} = P_{human} K_{assistance}$ calculation. For this, it is necessary to look at the largest positive value of the power-time graph; it must be equal to the $P_{motor} (1 + K_{assistance}) = P_{b-out}$ value. The resulting value must meet the nominal value of the motor to be selected. When calculating regen, our value symbolized by α gives us electrical efficiency. The α value is actually empirical and dynamic, therefore we must use the table in appendix 1.1 in the simulation.

Power Time Curve

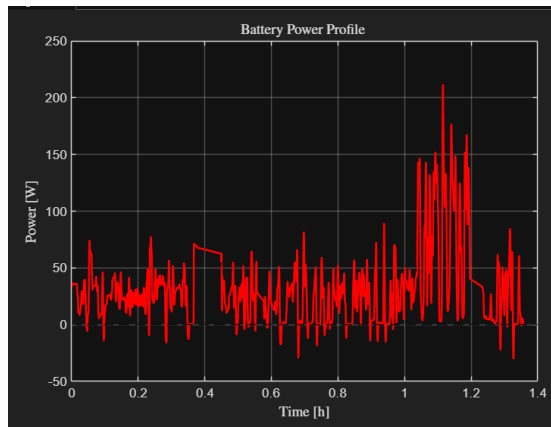


Figure 3 Power Time

This profile includes slow urban rides and intermediate accelerations, and there are actually two distinct riding styles. The first hour is generally flat and fast. Towards the end, there are power demands of 150-200W. The primary reasons for these sudden spikes in the graph may be GPS data and demand for hills and potholes. The area above the X-axis of this data allows us to calculate the bike's total energy needs, while the area below it allows us to calculate the Regen energy.

Component sizing

```
Analyzed Cycle Duration: 1.36 hours
Total Electrical Energy Required (Consumption Only):
-> 118652 Ws (Watt-seconds or Joules)
-> 32.96 Wh (Watt-hours)
-----
Total Energy Consumed (Wh): 32.9589
```

Figure 4 Total Power

We expect a range of 50 km from the bike, excluding regenerative braking. To achieve this range, the current riding cycle would need to be repeated for 13.92~14 hours. Due to the peaks

experienced with the motor, 250W is considered appropriate for regulation and motor health. Additionally, the calculations required for a battery range of 50 km are listed on the side. $\frac{32.96Wh}{4.81km} = 6.85Wh/km$ $6.85Wh/km \times 50km = 251.87Wh$ These values, which are classified as a typical mid-sized electric bike, are very close to the actual data.

Discussion & Future Works

These data directly affect our efficiency during regen. The result of the analyses turned out to be healthier than expected and has become meaningful data. It meets the criteria for a typical medium-sized e-bike design. During the regen phase, it is expected that the accuracy of the estimated data will be increased by taking the α constant, which comes into the formulas, as a dynamic value. α reacts according to the battery's current state of charge.

Appendices

Time definiton/Data Preparation/Cleaning/Manual Windowing/Measurement Windowing

	<pre>clear; close all; clc; % Bu betiği çalıştırdığınızdan emin olmak için dosya adını kontrol edin. fileName = 'E-Bike_Field_Study_Data_20251116 (1).csv'; % Dosya adını .xlsx'ten .csv'ye dönüştürdüm (Resimdeki koddan önce) % Eğer dosya .xlsx ise: fileName = 'E-Bike_Field_Study_Data_20251116 (1).xlsx';</pre>	
	<pre>%% 1) .CSV Dosyasını Oku % Veri okuma. İlk sütunları (date & time) string olarak, hız verisini % otomatik olarak okur. T = readtable(fileName, 'ReadVariableNames', false, 'Format', '%s%s%f');</pre>	
	<pre>% Değişken adları varsayılan olarak T.Var1, T.Var2, T.Var3 olacaktır. % Okuma işleminden sonra tablonun ilk iki satırını kontrol etmek % önemlidir. Resimdeki koddan önce bu satırlar silinmek istenmiştir.</pre>	
	<pre>%% 2) İlk satır başlık ise sil % Burada ilk satırın başlık mı yoksa veri mi olduğunu kontrol eden esnek bir mekanizma % oluşturulmuştur. Bu, verinin ham yapısına otomatik olarak uyum sağlar. if isequal(T.Properties.VariableNames, {'Var1', 'Var2', 'Var3'}) % Basit bir kontrol: İlk değişkenin değeri 'date' veya 'Var1' içeriyorsa başlık varsay. if contains(T.Var1{1}, 'date', 'IgnoreCase', true) disp('İlk satır başlık olarak algılandı ve silindi.');</pre>	
	<pre>T(1,:) = []; end end</pre>	
33	<pre>%% 4) Datetime Nesnesine Dönüştürme ve Hızın Çift Hassasiyetli Olmasını Sağlama</pre>	
34	<pre>% Tarih ve saat stringlerini, MATLAB'ın anladığı 'datetime' nesnesine çevirme.</pre>	
35	<pre>% 'datetime' nesneleri, zaman serisi işlemlerinde yüksek hassasiyet (nanosaniye) ve otomatik olarak sağlar.</pre>	
36	<pre>% YYYY-MM-DD hh:mm:ss.SSSSSS formatını varsayıyoruz (Resimdeki formatı baz alarak).</pre>	
37	<pre>% UTC (Zaman dilimi: +00:00) sorununu çözmek için '+00:00' yerine 'Z' eklenmesi daha genel bir yaklaşımdır.</pre>	
38	<pre>try</pre>	
39	<pre> fullTime = strcat(rawDateStr, 'Z'); % Stringe 'Z' (Zulu/UTC) ekleyerek zaman dilimi ataması.</pre>	
40	<pre> t = datetime(fullTime, 'InputFormat', 'yyyy-MM-dd HH:mm:ss.SSSSSSZ', 'TimeZone', 'UTC');</pre>	
41	<pre>catch</pre>	
42	<pre> warning('Varsayılan formatta hata! yyyy-MM-dd HH:mm:ss formatı deniyor.');</pre>	
43	<pre>% Eğer SSSSSS kısmı sorun çıkarırsa, daha kısa formatı dene.</pre>	
44	<pre> t = datetime(rawDateStr, 'InputFormat', 'yyyy-MM-dd HH:mm:ss', 'TimeZone', 'UTC');</pre>	
45	<pre>end</pre>	
46	<pre></pre>	
47	<pre>v_mph_raw = double(rawSpeed); % Hızı double yap (Zaten double okunmuş olabilir ama güvenlik için).</pre>	
48	<pre></pre>	
49	<pre></pre>	
50	<pre>%% 5) Veriyi Saniyelik Aralıklarla Örnekleme ve Ortalama Alma (Aggregation & Binning)</pre>	
51	<pre>% Ham verinin örnekleme hızı değişirse, veriyi sabit 1 saniye aralıklarla</pre>	
52	<pre>% ortalamak (aggregate) gerekir.</pre>	
53	<pre></pre>	
54	<pre>% Saniyelere yuvarla (Tüm veriyi 1 saniyelik zaman dilimlerine hizalar)</pre>	
55	<pre>uniqueTimes = dateshift(t, 'start', 'second');</pre>	

```

75 Hassasiyet = 0.1;
76 v_mean_mph_rounded = round(v_mean_mph_raw / Hassasiyet) * Hassasiyet;
77
78 %% 7) Aykırı Değerleri Azaltmak İçin HAREKETLİ ORTALAMA (Smoothing) Uygulama
79 % Hız sinyalindeki gürültüyü ve aykırı değerleri azaltmak için 3 saniyelik bir pencere ile yumuşatma
80 % Bu, fiziksel olarak daha makul ve görsel olarak daha temiz bir profil sağlar.
81 windowSize = 3;
82 v_mean_mph_smoothed = movmean(v_mean_mph_rounded, windowSize);
83
84 %% 8) Birim Dönüşümleri (mph -> km/h ve m/s)
85 % İşlenmiş hızı metrik birimlere çevirme (m/s ve km/h).
86 % 1 mph = 1.60934 km/h
87 % 1 mph = 0.44704 m/s
88
89 Conversion_mph_to_kmh = 1.60934;
90 Conversion_mph_to_mps = 0.44704;
91
92 v_mean_kmh = v_mean_mph_smoothed * Conversion_mph_to_kmh;
93 v_mean_mps = v_mean_mph_smoothed * Conversion_mph_to_mps;
94
95 %% 9) Grafiğe Hazırlık: Zamanı O'dan Başlat ve Saate Çevir
96 % Mutlak UTC zamanını, analiz için daha anlamlı olan başlangıçtan
97 % 'uniqueTimes' bir datetime vektörüdür. 'seconds' fonksiyonu, fa
98
99 % İlk zaman noktasını (uniqueTimes(1)) referans alarak farkı bul.
100 t_rel_seconds = seconds(uniqueTimes - uniqueTimes(1));
101 t_rel_hour = t_rel_seconds / 3600; % Saniyeden Saate Dönüşüm
102
103 %% 10) Veri Kırpma (Data Trimming)

```

 Snipping Tool

```

98 % Belirlenen limitin altındaki veya eşit olan zaman noktalarının endekslerini bul.
99 valid_indices = t_rel_hour <= limit_hour;
100
101 % Tüm ilgili değişkenleri bu endekslere göre kırp.
102 t_rel_hour_trimmed = t_rel_hour(valid_indices);
103 uniqueTimes_trimmed = uniqueTimes(valid_indices);
104 v_mean_kmh_trimmed = v_mean_kmh(valid_indices);
105 v_mean_mps_trimmed = v_mean_mps(valid_indices);
106
107 %% 11) İki Grafik Çizimi: km/h ve m/s
108 % Görüntüleme için iki alt grafik (subplot) oluşturma.
109 figure;
110 sgtitle(sprintf('E-Bike Sürüş Döngüsü Analizi (Kırpılmış: %.1f Saat)', limit_hour));
111
112 % Alt Grafik 1: Hız km/h cinsinden
113 subplot(2,1,1);
114 plot(t_rel_hour_trimmed, v_mean_kmh_trimmed, 'LineWidth', 1.5, 'Color', [0.85 0.33 0.1]);
115 grid on;
116 xlabel('Görelî Zaman [h]');
117 ylabel('Hız [km/h]');
118 title(sprintf('Hız Profili (%.1f Saat Limit, %.1f km/h Hassasiyet, %ds Yumusatma)', limit_hour, Hassas
119
120 % Alt Grafik 2: Hız m/s cinsinden
121 subplot(2,1,2);
122 plot(t_rel_hour_trimmed, v_mean_mps_trimmed, 'LineWidth', 1.5, 'Color', [0 0.45 0.74]);
123 grid on;
124 xlabel('Görelî Zaman [h]');
125 ylabel('Hız [m/s]');

```

```

123 grid on;
124 xlabel('Görelî Zaman [h]);
125 ylabel('Hız [m/s]);
126 title(sprintf('Hız Profili (Metrik Birimler)'));
127
128 %% 12) İşlenmiş Verinin Tablo Çıktısı (Driving Cycle Table)
129 % Analiz için kullanılan son, işlenmiş ve kırılmış verilerden bir tablo oluşturma.
130 % Bu tablo, modelleme veya daha ileri analizler için standart bir girdi olacaktır.
131
132 DrivingCycleTable = table(...
133     uniqueTimes_trimmed, ...
134     t_rel_hour_trimmed, ...
135     v_mean_kmh_trimmed, ...
136     v_mean_mps_trimmed, ...
137     'VariableNames', {'Time_UTC', 'Time_Hour', 'Speed_kmh_Smoothed', 'Speed_mps_Smoothed'});
138
139 % Tabloyu .mat dosyası olarak kaydetme
140 save('DrivingCycleTable.mat', 'DrivingCycleTable');
141
142 disp(' ');
143 disp('İşlenmiş Veriler DrivingCycleTable.mat dosyasına kaydedildi. ');
144 disp('Oluşturulan tablo değişkenlerinin ilk 5 satırını: ');
145 disp(head(DrivingCycleTable, 5));

```

Power graph

```
1 clear; close all; clc;
2
3 %% 0) LOAD TABLES
4 % Main driving cycle data (CONTAINS 3696 ROWS)
5 load DrivingCycleTable.mat
6 % SLOPE DATA GENERATED IN THE PREVIOUS STEP (CONTAINS 19788 ROWS)
7 try
8     load SlopeTable.mat % Loads a table named 'SlopeTable'
9 catch ME
10    error('SlopeTable.mat file not found. Please make sure you ran the previous script first.');
```

```
11 end
12
13 % Extract Variables
14 t_raw = seconds(DrivingCycleTable.Time - DrivingCycleTable.Time(1));
15 t = t_raw / 3600; % [h] Relative Time
16 V = DrivingCycleTable.Speed_mps; % [m/s] Speed
17
18 % --- NEWLY ADDED CODE: SIZE EQUALIZATION ---
19 n_driving_rows = length(V); % Size of the driving cycle (3696)
20 n_slope_rows = height(SlopeTable); % Size of the slope table (19788)
21
22 fprintf('Driving Cycle Size: %d rows\n', n_driving_rows);
23 fprintf('Slope Table Size: %d rows\n', n_slope_rows);
24
25 % Slope data cannot be shorter than driving data (Error check)
26 if n_slope_rows < n_driving_rows
27     error('Error: Slope data (%d) is shorter than the driving cycle (%d). Check data.', ...
28           n_slope_rows, n_driving_rows);
29 end
30
31 % LOGICAL SOLUTION: Take only the part of the slope data (19788) that
32 % matches the driving cycle (the first 3696 rows).
33 Slope_Factors_Full = SlopeTable.Slope;
34 slope_factors = Slope_Factors_Full(1:n_driving_rows); % TRIM THE VECTOR
35
36 fprintf('Size equalization complete: Slope data reduced to %d rows.\n', length(slope_factors));
37 % --- END OF NEWLY ADDED CODE ---
38
39 dt = 1; % [s] Time step
40
41 %% 1) Parameters - PHYSICAL MODEL
42 M = 125; % kg
43 g = 9.81; % m/s^2
44 Crr = 0.008; % Rolling Resistance Coefficient
45 rho = 1.225; % kg/m^3
46 Af = 1; % m^2
47 Cd = 0.85; % Drag Coefficient
48 Km = 1.05; % Rotary Mass Factor (phi)
49 nt = 0.8; % Drivetrain Efficiency (eta_t)
50 nm = 0.9; % Motor Efficiency (eta_m)
51 windowSize = 10; % s (Smoothing Window)
52
53 % --- DYNAMIC SLOPE CALCULATION ---
54 % (Now using the 'slope_factors' variable)
55 theta_magnitude = pi / 104.72;
56 theta = slope_factors * theta_magnitude; % [rad] DYNAMIC SLOPE VECTOR (3696x1 size)
57
58 %% 2) Acceleration (dV/dt) [m/s^2] - 10s Smoothing
59 a_raw = diff(V) / dt;
60 a_raw = [a_raw; a_raw(end)];
```

```

60     a_raw = [a_raw; a_raw(end)];
61     a = movmean(a_raw, windowSize);
62
63     %% 3) Force Components [N] - VECTORIZED CALCULATION
64     % Since all vectors (a, V, theta) are now 3696x1,
65     % these calculations will work without issues.
66     F_inertia = Km * M * a;
67     F_air     = 0.5 * Cd * rho * Af * V.^2;
68     F_hill    = M * g * sin(theta);
69     F_roll    = M * g * Crr .* cos(theta);
70     F_total   = F_inertia + F_air + F_hill + F_roll;
71
72     %% 4) Power Calculation and Smoothing
73     P_Mech_W = V .* F_total;
74     P_Battery_W_raw = P_Mech_W ./ (nt * nm);
75     P_Battery_W_smooth = movmean(P_Battery_W_raw, windowSize);
76     P_Battery_kW = P_Battery_W_smooth / 1000;
77
78     %% 5) Power Plot
79     figure;
80     plot(t, P_Battery_W_smooth, 'LineWidth', 1.5, 'Color', 'r');
81     grid on;
82     xlabel('Time [h]', 'Interpreter', 'latex');
83     ylabel('Power [W]', 'Interpreter', 'latex');
84     title('Battery Power Profile (Dynamic Slope - 3696s)', 'Interpreter', 'latex');
85     yline(0, 'k--', 'LineWidth', 1, 'HandleVisibility', 'off');
86
87     %% 6) Create Power Table and Output
88     PowerTable = table(t, DrivingCycleTable.Time, V, a, F_total, P_Mech_W, P_Battery_W_raw, P_Battery_W_smooth,
89         'VariableNames', {'Time_h', 'Time_raw_datetime', 'Speed_mps', 'Acceleration_mps2', 'Total_Force_N', ...
90             'P_Mech_W_Raw', 'P_Battery_W_Raw', 'P_Battery_W_Smooth', 'P_Battery_kW'});
91     disp(PowerTable);
92
93     % Updated the output filename to English as well
94     save('FinalPowerTable_DynamicSlope_3696.mat', 'PowerTable');
95     disp('Final Battery Power Table (FinalPowerTable_DynamicSlope_3696.mat) has been saved.');
```

Total Power Calc with model

1	%% E-Bike ENERGY CONSUMPTION ANALYSIS	
2	clear; close all; clc;	
3		
4	% Load the final data table containing the battery power profile.	
5	% --- GÜNCELLENEN SATIR ---	
6	% Önceki kodun çıktısı olan doğru dosyayı yüklüyoruz:	
7	try	
8	% (İngilizce sürümü kullandıysanız bunu seçin)	
9	load FinalPowerTable_DynamicSlope_3696.mat;	
10	catch ME	
11	try	
12	% (Türkçe sürümü kullandıysanız bunu seçin)	
13	load FinalPowerTable_DinamikEğim_3696.mat;	
14	catch ME_TR	
15	error('Hata: FinalPowerTable_...mat dosyası bulunamadı. Önceki betiği çalıştırdığınızdan emin olun.	
16	end	
17	end	
18	% --- GÜNCELLEME SONU ---	
19		
20	disp('Güç tablosu başarıyla yüklendi.');	
21		
22	%% 1) Data Retrieval and Time Step	
23	% Use the smoothed Battery Power (P_b-out) calculated in the previous script.	
24	P_smooth_W = PowerTable.P_Battery_W_Smooth; % [W] Smoothed Electrical Power	
25	dt_seconds = 1; % [s] Time step (Data is 1 Hz)	
26		
27	%% 2) Power to Energy Conversion (Integration)	
28	% Energy (E) is the integral of Power (P) over time (dt).	
29	% We only consider positive power (consumption) and ignore negative power (regeneration).	
30	P_consumption_W = P_smooth_W;	
31		
32	% Set all non-positive power values (braking/regeneration) to zero.	
33	P_consumption_W(P_consumption_W < 0) = 0;	
34		
35	% Total Energy in Watt-seconds (Ws) or Joules (J).	
36	Total_Energy_Ws = sum(P_consumption_W) * dt_seconds;	
37		
38	%% 3) Unit Conversion (Ws to Wh)	
39	% Convert from Watt-seconds to Watt-hours (1 hour = 3600 seconds).	
40	Total_Energy_Wh = Total_Energy_Ws / 3600;	
41		
42	%% 4) Results Output (Plain English)	
43	disp('--- TOPLAM ELEKTRİKSEL ENERJİ TÜKETİM ANALİZİ ---');	
44	% Display the total duration of the cycle	
45	fprintf('Analiz Edilen Döngü Süresi: %.2f saat\n', PowerTable.Time_h(end));	
46	% Display the calculated energy values	
47	fprintf('Gereken Toplam Elektriksel Enerji (Sadece Tüketim):\n');	
48	fprintf('-> %.0f Ws (Watt-saniye veya Joule)\n', Total_Energy_Ws);	
49	fprintf('-> %.2f Wh (Watt-saat)\n', Total_Energy_Wh);	
50	disp('-----');	
51		
52	% Final, simple consumption figure	
53	disp(['Toplam Tüketilen Enerji (Wh): ', num2str(Total_Energy_Wh)]);	

REFERENCES

- U.S. Department of Transportation. (t.y.). *E-Bike Field Study Data* [Veri seti]. Erişim: https://data.transportation.gov/Bicycles-and-Pedestrians/E-Bike-Field-Study-Data/xdkm-ken4/data_preview *Note: Analysis based on rider data coded "2022-09-04."*
- ÜNLÜSOY, Y. Samim. (t.y.). *ME 466 Performance of Road Vehicles* [Ders Sunumu]. Kurum: Middle East Technical University, Makine Mühendisliği Bölümü, Ankara.